COMP2511

Tutorial 4

Last Week's Tutorial

- Testing, Debugging, Exceptions
- Domain Modelling

This Week's Tutorial

- Design Principles
- Design by Contract
- Streams

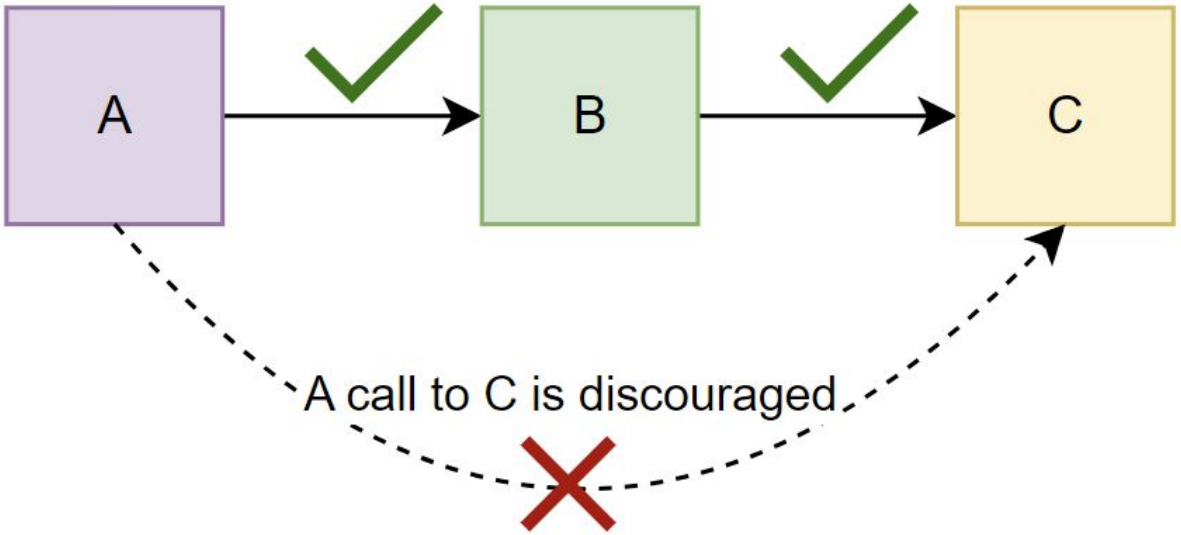
Design Principles

- Design principles are best-practice guidelines that help developers write object-oriented code that is understandable, flexible and maintainable.
- The **SOLID** Principles are the most common principles you will hear:
  - **Single Responsibility Principle**: Each class should have one specific role, and perform only that role.
  - **Open-Closed Principle**: New features can be added without modifying the existing code.
  - **Liskov Substitution Principle**: Subclasses should be valid instances of their superclass and can be substituted in place of the parent class without affecting correctness.
  - **Interface Segregation Principle**: Classes should never be forced to implement methods it will not use (think of abstract methods in abstract classes and interfaces).
  - **Dependency Inversion Principle**: All classes should interact with each other through abstractions.

The Law of Demeter/Principle of Least Knowledge

- Objects should only interact with their immediate dependencies and assume as little as possible about everything else in the system.
- This means that classes should only talk to its `friends', rather than any friends of friends.
- Violations of this principle *usually* look something like **`x.getA().getB()`**.
  - This is referred to as **message chaining**.
  - Changes to A or B would affect x directly, which increases **coupling**.
- In most Law of Demeter violations, the underlying issue is that responsibilities have not been handled in the appropriate classes (some classes not doing enough or some classes trying to do too much).

The Law of Demeter/Principle of Least Knowledge



The Law of Demeter/Principle of Least Knowledge

- To abide by the Law of Demeter, avoid using the methods of objects returned by another method.
  - e.g. if you are calling **`train.getStation()`**, you should avoid using Station methods in the same method.
- To fix Law of Demeter violations, make objects ask other objects to manipulate data, rather than asking for data and then manipulating it.
  - Doing **`A a = x.getA(); B b = a.getB()`** instead of **`x.getA().getB()`** is NOT a Law of Demeter fix.
  - Instead, ask A to manipulate B, then ask A for the overall result. Here, only immediate dependencies are used.

Quiz: Design Principles

- **For each example, which design principle is being violated? Think about how you would refactor the code to remove the violation.**
- **Example 1:** For assignment-i, a student has implemented all of the movement logic inside of the TrainsController's `simulate()` function. As such, their Train classes did not contain much code except for a few getter and setter functions.
- **Single Responsibility Principle**; the controller should only be involved with dealing with user input and invoking functionality on key classes like Train or Station. Move the movement logic into the Train/other classes so that movement becomes an abstraction from the perspective of the controller.

Quiz: Design Principles

- **Example 2:** Recall the Hotel exercise from Lab 2. Suppose the code for each type of room was handled in Room.java in the following manner:

```java
public class Room {
    private String roomType;

    public void printWelcomeMessage() {
        switch (roomType) {
            case "standard":
                System.out.println("Standard Room");
            case "ensuite":
                System.out.println("Ensuite Room");
            case "penthouse":
                System.out.println("Penthouse Room");
            case "budget":
                System.out.println("Budget Room");
        }
    }
```

```java
public double getPrice() {
    switch (roomType) {
        case "standard":
            return 100;
        case "ensuite":
            return 150;
        case "penthouse":
            return 200;
        case "budget":
            return 50;
    }
```

- **Open-Closed Principle**; adding a new or changing a room type affects the Room class. Make each room type a subclass with different behaviours.

Quiz: Design Principles

- **Example 3**: A student creates an `Animal` abstract class with the abstract methods **`eat()`**, **`move()`** and **`makeNoise()`**.
- **Interface Segregation Principle**; all animals eat, but not all animals move (e.g. starfish) or make noise (e.g. jellyfish). Separate **`move()`** and **`makeNoise()`** into different interfaces, e.g. `Mobile` and `Audible`.
- **Example 4**: A student makes `Square` a subclass of `Rectangle`. Rectangles have a length and width, getters and setters for each and a method to compute the area.
- **Liskov Substitution Principle**; setting the length/width of a square changes the other, which is unexpected behaviour for a rectangle. Do not make `Square` a subclass of `Rectangle`.

Design by Contract

- **Design by Contract (DbC)** is an approach where code is written around clear and well-defined rules and conditions, which *when adhered to* guarantees correct and expected behaviour.
    - For example, an assignment may specify that null will not be given as an input, so you do not have to account for this.
- This contrasts **defensive programming**, where erroneous inputs or user actions are checked and accounted for.
- What are the pros and cons of each approach?
    - **Design by Contract**: Easier to program and read, but less secure and robust.
    - **Defensive Programming**: More secure and robust, but harder to program and read.

Design by Contract

- In the context of a method,
  - a **precondition** is a condition on the **input** that the method will perform as expected.
  - a **postcondition** is a guarantee on what the method will **output** or do, *as long as the precondition holds*.
- If a method's precondition is not satisfied when it is called, you can no longer make any assumptions about the method's correctness (undefined behaviour).
- An **invariant** is a property of the system that is guaranteed to be maintained once any method has been performed.
  - Invariants can break *during* a method's execution, but must be restored before the end of the method.

Design by Contract and LSP

- The **Liskov Substitution Principle** (LSP) states that everywhere an instance of a superclass is used, an instance of its subclass should be able to be used in place of it.
- Contracts are inherited from superclasses to their subclasses, but can be modified in the subclass as long as LSP still holds.
  - **preconditions** can be **weakened** in the subclass; subclasses can accept a **wider** range of inputs, as long as all inputs that worked with the superclass still work.
    - **Example**. If a superclass accepts ints between 0-50, a subclass can accept 0-100.
  - **postconditions** can be **strengthened** in the subclass; subclasses can be more **specific** about its output, as long as it is an acceptable output for the superclass.
    - **Example**. If a superclass returns a list in any order, a subclass can return a sorted list.

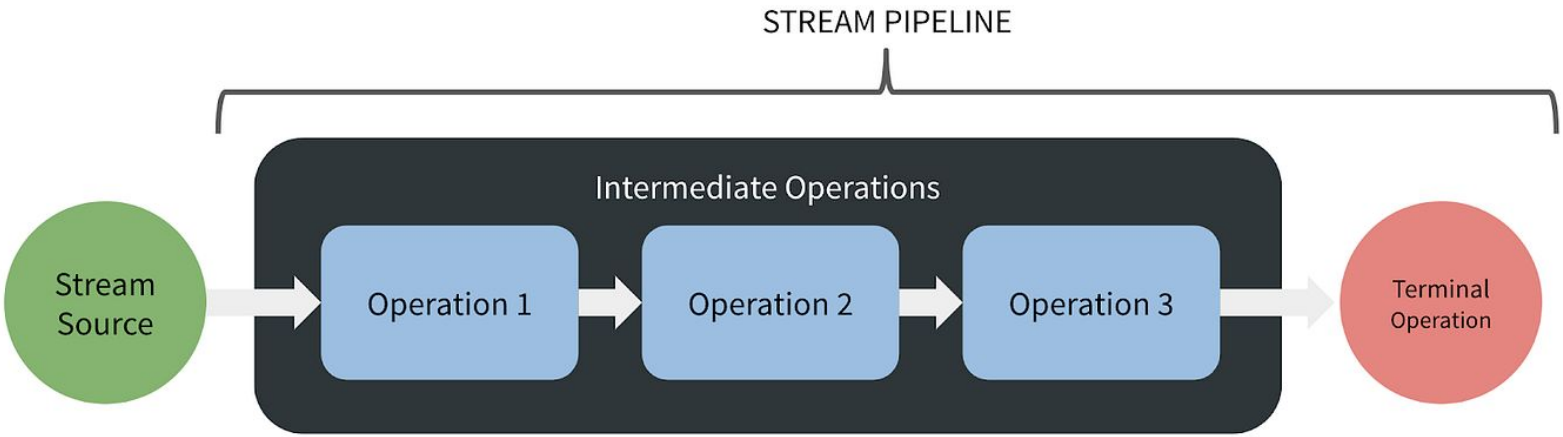
Live Example: Design by Contract and LSP

- Analyse the code in **src/preconditions** and **src/postconditions** and determine which subclasses are valid under LSP.

Streams

- A stream is an abstracted sequence of elements. Collections in Java (eg. **`List`**, **`Map`**, **`Set`**) can be transformed into streams using **`.stream()`**.
- Stream elements can only be used through specific methods that only work on streams and provide abstractions to common procedures.
- Two types of stream operations:
  - **Intermediate** operations: transforms the stream into another stream. These operations can be chained together.
    - **`map`**, **`filter`**, **`distinct`**
  - **Terminal** operations: marks the end of a stream pipeline and produces a final result.
    - **`reduce`**, **`sum`**, **`toList`**

Streams


STREAM PIPELINE
Intermediate Operations
Stream Source
Operation 1
Operation 2
Operation 3
Terminal Operation

Stream Operations

```
List<Integer> l = List.of(1, 2, 3, 4, 5, 6);
l = l.stream().filter(x -> x < 5).map(x -> x * 3).toList();
```

```
[3, 6, 9, 12]
```

- **`filter()`**: only keeps elements that satisfy a specific condition.
  - `.filter(x -> x < 5)` removes all elements greater than or equal to 5.
- **`map()`**: applies a function to all elements in the stream.
  - `.map(x -> x * 3)` multiplies all numbers in the stream by 3.
- **`toList()`**: converts stream into a list (returned list is **immutable**).
- **`reduce()`**: combines elements into a single result.
  - `.reduce(1, (a, b) -> a * b)` multiplies all numbers in the stream, starting with 1.
- **`distinct()`**: removes duplicate elements.

Live Example: Streams

- Complete the tasks in **src/stream** using streams.